



Alma Mater Studiorum-Università di Bologna Scuola di Ingegneria

Lambda expression

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



FUNZIONI NEI LINGUAGGI DI PROGRAMMAZIONE

- Tutti i linguaggi di programmazione hanno costrutti per esprimere *funzioni*, MA nei linguaggi imperativi tradizionali una funzione è *solo un costrutto che incapsula codice*
 - ha un nome, un'intestazione (signature), un corpo
 - viene *chiamata* con l'operatore `()`
- MOTIVO: tradizionalmente, il software suddivide a priori «dati» e «codice»
 - i dati stanno da una parte, il codice da un'altra
 - i dati sono «passivi», il codice «attivo»
- MA.. *è veramente «giusta», «sana», questa distinzione?*



FUNZIONI NEI LINGUAGGI DI PROGRAMMAZIONE

- Vedere la funzione solo come *solo un (mero) costrutto che incapsula codice* corrisponde una visione «low level», ispirata dall'hardware e dai linguaggi assembly
- È una visione miope, peraltro *difforme dalla matematica*
 - ESEMPLI: $f1: R \rightarrow R, x \rightarrow x+3$ $f2: R \times R \rightarrow R, x, y \rightarrow x+\pi y$
- Perché *non* interpretare la funzione, piuttosto, come *un particolare «tipo di dato»*?
 - con le sue proprietà (nome, intestazione, argomenti, codice), accessibili magari tramite i suoi accessor
 - definibile sia come literal (costante scritta nel codice), sia, magari, perfino dinamicamente (tramite **new**)
 - *un dato con un'unica vera caratteristica in più: essere eseguibile*



FUNZIONI NEI LINGUAGGI DI PROGRAMMAZIONE

- Non considerare la funzione come un particolare tipo di dato, come accade nei linguaggi tradizionali, ha conseguenze:
 - *non può essere assegnata a variabili*
 - approssimazione: i puntatori a funzione del C, che però contengono solo l'indirizzo del codice, non una vera "entità funzione"..
 - *non può essere passata come argomento a un'altra funzione*
 - perché gli argomenti devono essere *valori* di un qualche *tipo*, mentre le funzioni in quei linguaggi non lo sono
 - *non può essere restituita da un'altra funzione*
 - perché non si può "sintetizzare comportamento" come fosse un valore (e le funzioni esprimono "comportamenti"...)
 - al più, si può *restituire l'indirizzo di una funzione già esistente*
 - *non può essere definita e usata "al volo", tramite **new***
 - le funzioni devono essere definite nel codice, staticamente



FUNZIONI

NEI LINGUAGGI FUNZIONALI

- Al contrario, i linguaggi *funzionali* si basano proprio sull'idea di *funzioni* come *fondamentale tipo di dato*
 - si usa dire come «first-class entities»
- Una funzione che sia *first-class entity* deve essere *manipolabile come ogni altro tipo di dato* e quindi:
 - deve poter essere *assegnata a variabili*, di tipo "funzione da A a B"
 - deve poter essere *passata come argomento* a un'altra funzione
 - deve poter essere *definibile come ogni altro oggetto*
- Diviene quindi possibile *passare una funzione (ossia, «comportamento»)* ad altre funzioni, come argomento
 - con la stessa facilità con cui passiamo *dati*, in questa prospettiva possiamo pensare di *passare un'entità eseguibile!*



PASSARE «COMPORTAMENTO»

- Poter *passare una funzione («comportamento»)* ad altre funzioni è utile in molte situazioni concrete:
 - per *rendere parametrici* gli algoritmi generali
→ caso tipico: i comparatori negli algoritmi di ordinamento
 - per passare *funzioni di callback* con cui esprimere azioni da svolgere successivamente, «nel caso in cui..»
→ caso tipico: la reazione agli eventi nella grafica
 - per *riprogettare in modo più naturale librerie ed API di largo uso*
 - ..e in molte altre situazioni!
- Senza questa tecnologia, tali situazioni sono complesse (e spesso contorte!) da esprimere
→ per questo, *oggi tutti le incorporano!*



LINGUAGGI A OGGETTI «BLENDED»

- Molti linguaggi a oggetti, nati su base imperativa, *stanno incorporando il concetto di funzione come first-class entity* [o qualcosa che le somigli] → **linguaggi *blended***
 - **Javascript**: da sempre ha il costruttore **Function** e la keyword **function** per specificare funzioni "anonime"
 - **Java**: supporto *parziale* introdotto in Java 8
 - funzioni (anonime) come *lambda expression*
 - si possono definire come literal e passare come argomenti
 - MA non sono vere first class entities, perché *non definiscono un tipo*
 - anziché appartenere a un vero "tipo-funzione", si appoggiano a un "opportune interfacce, dette *interfacce funzionali*"
 - **C#**: supporto efficace, tramite *delegati*
 - **Kotlin, Scala**: supporto *completo* molto efficace, API riorganizzate



FUNZIONI COME VERI OGGETTI

- Cosa significa avere «funzioni come veri oggetti»?
 - le funzioni devono essere viste come *istanze* di un opportuno *tipo*
 - si devono poter definire *riferimenti* (a funzioni) di quel *tipo*
 - si devono poter creare *oggetti* (funzioni) di quel *tipo*
 - si devono poter *assegnare* tali oggetti (funzioni) a riferimenti di quel tipo
 - si devono poter *passare come argomenti* ad altre funzioni
 - in breve, si devono poter trattare in modo *uniforme a ogni altro oggetto*, con l'unica caratteristica ulteriore di essere *eseguibili*: oggetti su cui applicare l'*operatore di chiamata* `()` che *mette in esecuzione il codice*

Non tutti i linguaggi a oggetti *blended* supportano *tutte* queste caratteristiche in modo *completo*.

LINGUAGGI BLENDED

- I vari blend «at a glance»

- funzioni viste come *istanze* di un *tipo-funzione*
- definire *referimenti* (a funzioni) di quel *tipo*
- creare *oggetti* (funzioni) di quel *tipo*
- *assegnare* tali oggetti a riferimenti
- *passare come argomenti* ad altre funzioni
- applicare l'*operatore di chiamata* ()

Java	C#	Scala	Kotlin
Java	C#	Scala	Kotlin
Java	C#	Scala	Kotlin
Java	C#	Scala	Kotlin
Java	C#	Scala	Kotlin
	C#	Scala	Kotlin

- Java è un blend soltanto parziale

- supporto introdotto in Java 8
- non introduce un vero *tipo-funzione*, ri-sfrutta la nozione di *interfaccia*
- *non consente* di applicare direttamente l'operatore di chiamata ()



IL BLEND DI JAVA

- Java è arrivato *buon ultimo* a proporre il suo "blend"
- Obiettivi primari:
 - retrocompatibilità, più che innovazione totale
 - nessuno stravolgimento del linguaggio
(in particolare: mantenimento della nozione di *nominal typing*)
- Risultato: un blend *all'80%*
 - esistono oggetti-funzione, ma *non esiste un vero "tipo funzione"*:
si sfrutta a tal fine la nozione di interfaccia → *interfacce funzionali* ☹️
 - gli oggetti-funzione *non* sono *direttamente* eseguibili con l'operatore di chiamata `()`: occorre *invocare per nome il metodo sottostante* ☹️ ☹️ ☹️
- Massima resa, ma risultato "non fully functional"



LAMBDA EXPRESSION

- Per definire oggetti-funzione si utilizza un *particolare tipo di espressione*, noto come *lambda expression*
- Ricorda la notazione delle funzioni in matematica:

$$x \rightarrow x+3 \qquad x,y \rightarrow 2x-y$$

Una *lambda expression* è una *funzione anonima*, espressa in modo simile alla notazione matematica:

Java

(lista argomenti) -> { corpo della funzione }

Le parentesi tonde/graffe si possono *omettere* nei casi semplici:

- se c'è un solo statement nel corpo $\rightarrow$ si omettono le *graffe*
- se c'è un solo argomento in lista $\rightarrow$ si omettono le *tonde*

I *tipi degli argomenti* si possono omettere se deducibili dal contesto



LAMBDA EXPRESSION NEI VARI LINGUAGGI

- Java:

(lista argomenti) -> { corpo della funzione }

Java

- C#

(lista argomenti) => { corpo della funzione }

C#

- Scala:

(lista argomenti) => { corpo della funzione }

Scala

- Kotlin:

{ lista argomenti -> corpo della funzione }

Kotlin

- NB: da un linguaggio all'altro cambiano il *tipo di freccia* e alcuni dettagli sulla collocazione e l'obbligatorietà delle varie *parentesi*, oltre ovviamente al modo di specificare il *tipo* degli argomenti

IL TIPO DELLA LAMBDA

- In matematica, le funzioni si definiscono stabilendo *dominio* e *codominio*:

$$f: \mathbb{Z} \rightarrow \mathbb{Z}$$

$$x \rightarrow x+3$$

$$g: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$$

$$x, y \rightarrow 2x-y$$

$$h: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$$

$$x, y \rightarrow x + \pi y$$

- Dunque, *le funzioni non hanno tutte lo stesso «tipo»*
 - una funzione sul dominio $\mathbb{Z}$ non è intercambiabile con una su $\mathbb{R} \times \mathbb{R}$
 - il «tipo» della funzione è definito dal suo *dominio* e dal suo *codominio*

Ad esempio, nei casi sopra:

– $f: \mathbb{Z} \rightarrow \mathbb{Z}$ «funzione a un argomento dagli interi agli interi»

– $h: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ «funzione a due argomenti dai reali ai reali»

- Ciò richiede di *specificare il tipo* di argomenti e risultato

IL TIPO DELLA LAMBDA

- Nella lista argomenti occorre perciò, di norma, specificare il *tipo* di ogni argomento
 - salvo che si possa dedurlo dal contesto tramite *type inference*
- Il tipo del risultato è spesso dedotto per *type inference*
 - ESEMPIO: funzione che, dati $(x,y) \in \mathbb{Z}$, calcola $2x-y$

```
(int x, int y) -> 2*x-y
```

Java

```
(int x, int y) => 2*x-y
```

C#

```
(x:Int, y:Int) => 2*x-y
```

Scala

```
{ x:Int, y:Int -> 2*x-y }
```

Kotlin

- Scrivere un siffatto *lambda literal* **equivale a fare la new di un nuovo oggetto-funzione**, istanza di un "*opportuno tipo*" stabilito dal compilatore (o dichiarato dall'utente)



IL TIPO DELLA LAMBDA

- Come insegna la matematica, ogni funzione ha il suo *tipo* che specifica o riassume *dominio e codominio*
- Ad esempio:

`(int x, int y) -> 2*x-y` $f: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$

«funzione a due argomenti dagli interi agli interi»

`(Counter c) -> c.getValue()` $f: C \rightarrow \mathbb{N}$

«funzione a un argomento dall'insieme dei Counter ai naturali»

`(String s, int n) -> s+n` $f: S \times \mathbb{Z} \rightarrow S$

«funzione a due argomenti, una stringa e un intero, a stringa»

eccetera

- *Come specificare ciò* in un linguaggio di programmazione?



TIPO NOMINALE vs. STRUTTURALE

- Posto che il *tipo della funzione* è una locuzione come:
 - $f: Z \rightarrow Z$ «funzione a un argomento dagli interi agli interi»
 - $h: R \times R \rightarrow R$ «funzione a due argomenti dai reali ai reali»
- Ci sono due possibilità:
 - *dare un nome* a ogni tipo $\rightarrow$ approccio **NOMINALE**
 - *scrivere «da dominio a codominio»* $\rightarrow$ approccio **STRUTTURALE**
- Entrambi gli approcci hanno pro e contro:
 - **l'approccio nominale** costringe a inventarsi nomi per ogni possibile tipo di funzione 😞, MA è poco invasivo in un linguaggio esistente 😊
 - **l'approccio strutturale** evita di doversi inventare altri nomi 😊, MA si incastra più in profondità nel type system di un linguaggio 😞 rendendo non semplice la sua adozione in linguaggi già esistenti 😞



TIPO NOMINALE vs. STRUTTURALE

- Con il *tipo nominale* bisogna inventarsi dei *nomi* per le diverse possibili situazioni, ad esempio:
 - $f: Z \rightarrow Z$ diremo che è di tipo *IntToIntFunction* (Java) o magari *Func<int,int>* (C#)
 - $h: R \times R \rightarrow R$ diremo che è di tipo *RealToRealBiFunction* (Java) o magari *Func<double,double,double>* (C#)
- Con il *tipo strutturale* la notazione stessa funge da tipo:
 - $f: Z \rightarrow Z$ diremo che è di tipo *« $Z \rightarrow Z$ »* (o notazione analoga)
 - $h: R \times R \rightarrow R$ diremo che è di tipo *« $R \times R \rightarrow R$ »* (o notazione analoga)
- Ci sono pro e contro in entrambe le scelte
 - l'approccio strutturale è la soluzione più pulita se *si parte da zero*
 - l'approccio nominale è più semplice da inserire *in un linguaggio col type system già definito*, ma implica definire e ricordare (molti) nomi



LE SCELTE DEI VARI LINGUAGGI

- Java e C# scelgono l'approccio *nominale* C# Java
 - MOTIVO: retrocompatibilità, facilità di riconversione del legacy code
 - PRO: si sposa *naturalmente* con il concetto di *interfaccia* (eventualmente adattato, v. «*delegato*» in C#) → integrazione seamless 😊
 - CONTRO: necessità di *definire* e *ricordare* parecchie interfacce 😞
(*interfacce funzionali* in Java) o *delegati standard* (C#)
- Scala, Kotlin optano invece per l'approccio *strutturale* Scala Kotlin
 - MOTIVO: sono stati definiti incorporando *fin dall'inizio* la nozione di funzione come «first-class entity» nel type system del linguaggio 😊
 - PRO: il tipo della lambda è evidente, non occorre ricordarsi nomi
 - CONTRO: type system più complesso, leggibilità.. *dipende* 😊

IL TIPO DELLA LAMBDA NEI VARI LINGUAGGI

- In Java e C# il tipo della lambda è dunque espresso da un *nome* che «evochi» dominio e codominio della funzione
 - in Java, da un'appropriata *interfaccia funzionale* scelta fra quelle standard definite nel package `java.util.function`
 - in C#, da uno dei *delegati standard* `Func<...>` o `Action<...>`
- Nel caso dei nostri due esempi:

– $f: Z \rightarrow Z$	in Java	f è di tipo <code>IntUnaryOperator</code>	Java
	in C#	f è di tipo <code>Func<int,int></code>	C#
	in Scala	f è di tipo <code>Int => Int</code>	Scala
	in Kotlin	f è di tipo <code>Int -> Int</code>	Kotlin
– $h: R \times R \rightarrow R$	in Java	h è di tipo <code>DoubleBinaryOperator</code>	Java
	in C#	h è di tipo <code>Func<double,double,double></code>	C#
	in Scala	h è di tipo <code>Double, Double => Double</code>	Scala
	in Kotlin	h è di tipo <code>Double, Double -> Double</code>	Kotlin



ESEMPI MULTI-LINGUAGGIO

- Dichiarazioni complete con tutti i tipi specificati:

```
IntUnaryOperator f = (int x) -> x+3;  
Func<int,int>    f = (int x) => x+3;  
val f : Int => Int = x:Int => x+3;  
val f : (Int) -> Int = { x:Int -> x+3; }
```

Java

C#

Scala

Kotlin

- Dichiarazioni abbreviate con type inference:

```
IntUnaryOperator f = x -> x+3;  
Func<int,int>    f = x => x+3;  
val f : Int => Int = x => x+3;  
val f = x:Int => x+3;  
val f : (Int) -> Int = { x -> x+3; }  
val f = { x:Int -> x+3; }
```

Java

C#

Scala

Kotlin



LAMBDA CON «VOID»

- Dubbio: *dominio o codominio* potrebbero essere il «vuoto»?
- Sì, perché hanno entrambi senso:
 - *dominio vuoto* rappresenta *funzioni che generano valori*
 - in Java, il loro tipo è la specifica interfaccia funzionale *Supplier*
 - in C#, il loro tipo è espresso normalmente da **Func<...>**, avente come *unico argomento il tipo del risultato*
 - in Scala e Kotlin, il loro tipo è espresso semplicemente da **() -> ...**
 - *codominio vuoto* rappresenta *procedure che non restituiscono nulla*
 - in Java, il loro tipo è la specifica interfaccia funzionale *Consumer*
 - in C#, il loro tipo è espresso dal delegato **Action<...>**, avente come *unici argomenti i tipi del dominio*
 - in Scala e Kotlin, il tipo del risultato è **Unit**

ESEMPI MULTI-LINGUAGGIO

- Funzioni con dominio vuoto

$f: \text{void} \rightarrow \mathbb{R}$ «funzione che genera un valore reale»

```
DoubleSupplier f = () -> Math.sqrt(2);
```

Java

```
Func<double> f = () => Math.Sqrt(2);
```

C#

```
val f : () => Double = () => Math.sqrt(2);
```

Scala

```
val f : () -> Double = { Math.sqrt(2.0); }
```

Kotlin

- Funzioni con codominio vuoto

$p: S \rightarrow \text{void}$ «procedura che, data una stringa, la stampa»

```
Consumer<String> p = s -> System.out.println(s);
```

Java

```
Action<string> p = (s) => Console.WriteLine(s);
```

C#

```
val p : String => Unit = (s) => println(s);
```

Scala

```
val p : (String) -> Unit = { s -> println(s); }
```

Kotlin



CHIAMARE UNA FUNZIONE LAMBDA

- Dopo essere stata *definita*, una funzione (lambda) deve anche poter essere *usata*
 - *in matematica*, una funzione viene «*applicata*» ad argomenti scrivendone il nome seguito dagli argomenti fra parentesi
 - ad esempio, se $f = x^2 + 2x + 1$ (definizione), la sua applicazione all'argomento 3 si scrive solitamente come **$f(3)$** e vale $3^2 + 2 \times 3 + 1$
 - *nei linguaggi di programmazione*, una funzione viene «*chiamata*» circa con la stessa sintassi
 - ad esempio, la funzione sopra si può definire scrivendo `double f(double x){ return x*x+2*x+1; }` e chiamare con `f(3)`
 - *una lambda expression* dovrebbe poter seguire la stessa sintassi, **ma questo non è sempre vero**
 - in particolare *non è vero in Java*, perché le lambda in tale linguaggio *non sono vere «first class entities»* !



CHIAMARE UNA FUNZIONE LAMBDA

- Una lambda expression
 - può essere chiamata in modo analogo a una funzione «classica», ossia scrivendone il nome seguito dagli argomenti fra parentesi, *se il linguaggio la vede come first-class entity*
 - è il caso di `C#` `Scala` `Kotlin`
 - *deve invece essere chiamata ricorrendo a qualche circonvoluzione se il linguaggio non ha tale nozione* (ma la approssima soltanto)
 - è il caso di `Java` che deve appoggiarsi a un *metodo ausiliario*
 - MOTIVO: una lambda expression in Java è in realtà un'istanza di una opportuna interfaccia, che internamente dichiara un normale metodo: per chiamare la funzione occorre quindi invocare tale metodo
 - ad esempio, una funzione $f: R \rightarrow R$ è di tipo `DoubleUnaryOperator`, un'interfaccia che dichiara internamente il metodo `applyAsDouble`: quindi per chiamare $f(3)$ si deve scrivere `f.applyAsDouble(3)`



CHIAMARE UNA LAMBDA

ESEMPI MULTI-LINGUAGGIO

- Dichiarazioni complete con tutti i tipi specificati:

```
IntUnaryOperator f = (int x) -> x+3;  
Func<int,int>     f = (int x) => x+3;  
val f : Int => Int = x:Int => x+3;  
val f : (Int) -> Int = { x:Int -> x+3; }
```

Java

C#

Scala

Kotlin

- Esempi di chiamate (funzione da int a int):

```
int res = f.applyAsInt(3);  
int res = f(3);  
val res = f(3);  
val res = f(3);
```

Java

C#

Scala

Kotlin

- purtroppo, in Java il nome del metodo interno all'interfaccia non è neppure fisso: *varia da un'interfaccia funzionale all'altra*

ESEMPI MULTI-LINGUAGGIO

```
import java.util.function.*;

public class LambdaNew1 {
    public static void main(String[] args) {

        IntUnaryOperator f = (int x) -> x+3;
        IntBinaryOperator g = (int x, int y) -> 2*x-y;
        DoubleBinaryOperator h = (double x, double y) -> x+y*Math.PI;
        System.out.println("f(4) = " + f.applyAsInt(4));           // 7
        System.out.println("g(3,4) = " + g.applyAsInt(3,4));       // 2
        System.out.println("h(Math.PI,1) = " + h.applyAsDouble(Math.PI,1)); // 2*PI
    }
}
```

Java

JAVA: la lambda ha un tipo *nominale* espresso da un'interfaccia funzionale. Inoltre, non si può usare direttamente l'operatore di chiamata ()

C#: la lambda ha un tipo *nominale* espresso dal *delegato* Func<...>. Si può usare l'operatore di chiamata ()

```
public static void Main()
{
    Func<int,int> f = (int x) => x+3;
    Func<int,int,int> g = (int x, int y) => 2*x-y;
    Func<double,double,double> h = (double x, double y) => x+y*Math.PI;
    Console.WriteLine("f(4) = " + f(4));           // 7
    Console.WriteLine("g(3,4) = " + g(3,4));       // 2
    Console.WriteLine("h(Math.PI,1) = " + h(Math.PI,1)); // 2*PI
}
```

C#

```
def main(args:Array[String]):Unit = {

    var f = (x:Int) => x+3;
    var g = (x:Int, y:Int) => 2*x-y;
    var h = (x:Double, y:Double) => x+y*Math.PI;
    println("f(4) = " + f(4));           // 7
    println("g(3,4) = " + g(3,4));       // 2
    println("h(Math.PI,1) = " + h(Math.PI,1)); // 2*PI
}
```

Scala

```
fun main() {
    var f = {x:Int -> x+3};
    var g = {x:Int, y:Int -> 2*x-y};
    var h = {x:Double, y:Double -> x+y*Math.PI};
    println("f(4) = " + f(4));           // 7
    println("g(3,4) = " + g(3,4));       // 2
    println("h(Math.PI,1) = " + h(Math.PI,1.0)); // 2*PI
}
```

Kotlin

Scala, Kotlin: la lambda ha un tipo *strutturale* deducibile anche dalla *type inference*

Si può usare l'operatore di chiamata ()



INTERFACCE FUNZIONALI JAVA vs. DELEGATI C#

- Una *interfaccia funzionale* è una *normale interfaccia* che dichiara **un solo metodo**
 - si riconosce dall'annotazione **@FunctionalInterface**
 - interfaccia «SAM» = *Single Abstract Method*
 - il metodo dichiarato è quello che *incapsulerà il codice* della funzione
 - per chiamare la funzione occorrerà *chiamare quel metodo*, di cui quindi **occorre conoscere il nome** ☹ → **impossibile uso diretto di ()**
 - Java ne definisce moltissime (40+), che occorre conoscere ☹
- Un *delegato* è *simile* a un'interfaccia, ma è adattato e reso *specifico per dichiarare tipi-funzione*
 - internamente anche qui un metodo incapsula il codice della funzione, **ma ciò non traspare esternamente** → **possibile uso diretto di ()**
 - C# ne definisce 3 standard, che coprono il 99% dei casi pratici ☺

Java

C#



JAVA: FUNCTIONAL INTERFACES

- Il package `java.util.function` definisce *più di 40 interfacce funzionali* per tutti i casi di più largo uso

Package `java.util.function`

Java

Functional interfaces provide target types for lambda expressions and method references.

Functional interfaces provide target types for lambda expressions and method references. Each functional interface has a single abstract method, called the *functional method* for that functional interface, to which the lambda expression's parameter and return types are matched or adapted. Functional interfaces can provide a target type in multiple contexts, such as assignment context, method invocation, or cast context:

- il nome del metodo interno tende a "richiamare l'uso tipico" della funzione (con una certa fantasia..)
 - per le funzioni *da A a B* («*function*»), solitamente il metodo si chiama **apply** + varianti per tipi primitivi (**applyAsTipo**)
 - per le funzioni *da void a B* («*supplier*»), solitamente il metodo si chiama **get** + varianti per tipi primitivi (**getAsTipo**)
 - per le funzioni *da A a void* («*consumer*»), solitamente il metodo è **accept** (curiosamente, *senza* varianti per tipi primitivi: il nome è sempre **accept!**)



JAVA: FUNCTIONAL INTERFACES

Consumer
Supplier
Predicate
Function

BiConsumer
BiPredicate
BiFunction

UnaryOperator
BinaryOperator

BooleanSupplier

Versione singola
per **boolean**

DoubleConsumer
DoubleSupplier
DoublePredicate
DoubleFunction

DoubleUnaryOperator
DoubleBinaryOperator

DoubleToIntFunction
DoubleToLongFunction

ToDoubleFunction
ToDoubleBiFunction

ObjDoubleConsumer

Versioni specializzate
per **double**

IntConsumer
IntSupplier
IntPredicate
IntFunction

IntUnaryOperator
IntBinaryOperator

IntToDoubleFunction
IntToLongFunction

ToIntFunction
ToIntBiFunction

ObjIntConsumer

Versioni specializzate
per **int**

LongConsumer
LongSupplier
LongPredicate
LongFunction

LongUnaryOperator
LongBinaryOperator

LongToDoubleFunction
LongToIntFunction

ToLongFunction
ToLongBiFunction

ObjLongConsumer

Versioni specializzate
per **long**

**OSSERVA: la presenza dei tipi primitivi ci costa il 377% in più
in termini di interfacce da definire e ricordare!!**



JAVA: FUNCTIONAL INTERFACES

Consumer

Supplier

Predicate

Function

BiConsumer

BiPredicate

BiFunction

UnaryOperator

BinaryOperator

DoubleConsumer

DoubleSupplier

I consumatori sono funzioni a un argomento, che non restituiscono nulla

DoubleUnaryOperator

DoubleBinaryOperator

DoubleToIntFunction

DoubleToLongFunction

ToDoubleFunction

ToDoubleBiFunction

BooleanSupplier

ObjDoubleConsumer

IntConsumer

IntSupplier

IntPredicate

IntFunction

IntUnaryOperator

IntBinaryOperator

IntToDoubleFunction

Questi consumatori extra però sono funzioni a due argomenti. Anch'essi non restituiscono nulla

ObjIntConsumer

LongConsumer

LongSupplier

LongPredicate

LongFunction

LongUnaryOperator

LongBinaryOperator

LongToDoubleFunction

LongToIntFunction

ToLongFunction

ToLongBiFunction

ObjLongConsumer



JAVA: FUNCTIONAL INTERFACES

Consumer
Supplier
Predicate
Function

BiConsumer
BiPredicate
BiFunction

UnaryOperator
BinaryOperator

BooleanSupplier

DoubleConsumer
DoubleSupplier

**I fornitori sono funzioni
con zero argomenti che
producono un valore**

DoubleUnaryOperator
DoubleBinaryOperator

DoubleToIntFunction
DoubleToLongFunction

ToDoubleFunction
ToDoubleBiFunction

ObjDoubleConsumer

IntConsumer
IntSupplier
IntPredicate
IntFunction

IntUnaryOperator
IntBinaryOperator

IntToDoubleFunction
IntToLongFunction

ToIntFunction
ToIntBiFunction

ObjIntConsumer

LongConsumer
LongSupplier
LongPredicate
LongFunction

LongUnaryOperator
LongBinaryOperator

LongToDoubleFunction
LongToIntFunction

ToLongFunction
ToLongBiFunction

ObjLongConsumer



JAVA: FUNCTIONAL INTERFACES

Consumer

Supplier

Predicate

Function

BiConsumer

BiPredicate

BiFunction

UnaryOperator

BinaryOperator

DoubleConsumer

DoubleSupplier

DoublePredicate

**I predicati sono funzioni
con un argomento che
producono un boolean**

DoubleUnaryOperator

DoubleBinaryOperator

DoubleToIntFunction

DoubleToLongFunction

ToDoubleFunction

ToDoubleBiFunction

BooleanSupplier

ObjDoubleConsumer

IntConsumer

IntSupplier

IntPredicate

IntFunction

IntUnaryOperator

IntBinaryOperator

IntToDoubleFunction

IntToLongFunction

ToIntFunction

ToIntBiFunction

ObjIntConsumer

LongConsumer

LongSupplier

LongPredicate

LongFunction

LongUnaryOperator

LongBinaryOperator

LongToDoubleFunction

LongToIntFunction

ToLongFunction

ToLongBiFunction

ObjLongConsumer



JAVA: FUNCTIONAL INTERFACES

Consumer
Supplier
Predicate
Function

DoubleConsumer
DoubleSupplier
DoublePredicate
DoubleFunction

Versioni specializzate dal dominio primitivo
(**double, int, long**) a un codominio R

IntFunction

LongFunction

BiConsumer
BiPredicate
BiFunction

Le **funzioni** sono funzioni
generiche a *un argomento*, da
un dominio T a un codominio R

UnaryOperator
BinaryOperator

DoubleUnaryOperator
DoubleBinaryOperator

IntUnaryOperator
IntBinaryOperator

Versioni specializzate da dominio a
codominio primitivo (**double, int, long**)

DoubleToIntFunction
DoubleToLongFunction

IntToDoubleFunction
IntToLongFunction

LongToDoubleFunction
LongToIntFunction

ToDoubleFunction
ToDoubleBiFunction

ToLongFunction
ToLongBiFunction

ToLongFunction
ToLongBiFunction

BooleanSupplier

ObjDoubleConsumer

Versioni specializzate dal dominio T a un
codominio primitivo (**double, int, long**)

BooleanSupplier
ObjDoubleConsumer

NB: non sono previste interfacce funzionali standard per funzioni a più di due argomenti;
nel caso, dovranno essere definite dall'utente.



JAVA: FUNCTIONAL INTERFACES

Consumer
Supplier
Predicate
Function

BiConsumer
BiPredicate
BiFunction

UnaryOperator
BinaryOperator

I **biconsumatori** sono funzioni a *due argomenti* da un dominio TxU *che non restituiscono nulla*

I **bipredicati** sono funzioni a *due argomenti* da un dominio TxU *che producono un boolean*

Le **bifunzioni** sono funzioni generiche a *due argomenti*, da un dominio TxU a un codominio R

Consumer
Supplier
Predicate
Function
LongConsumer
LongSupplier
LongPredicate
LongFunction

UnaryOperator
BinaryOperator
DoubleFunction
LongUnaryOperator
LongBinaryOperator
LongToDoubleFunction
LongToIntFunction

ToDoubleFunction
ToLongFunction
ToDoubleBiFunction
ToIntBiFunction

BooleanSupplier
ObjDoubleConsumer

ObjIntConsumer

Versioni specializzate dal dominio TxU a un codominio primitivo (**double, int, long**)

NB: non sono previste interfacce funzionali standard per predicati a più di due argomenti; nel caso, dovranno essere definite dall'utente.



JAVA: FUNCTIONAL INTERFACES

Consumer	DoubleConsumer	IntConsumer	LongConsumer
Supplier	DoubleSupplier	IntSupplier	LongSupplier
Predicate	DoublePredicate	IntPredicate	LongPredicate
Function	DoubleFunction	IntFunction	LongFunction

BiConsumer
BiPredicate
BiFunction

Gli **operatori unari** sono
funzioni a *un argomento* su un
dominio T

Versioni specializzate su domini
primitivi (**double, int, long**)

UnaryOperator	DoubleUnaryOperator	IntUnaryOperator	LongUnaryOperator
BinaryOperator	DoubleBinaryOperator	IntBinaryOperator	LongBinaryOperator

Gli **operatori binari** sono
funzioni a *due argomenti* su un
dominio T

Gli operatori binari sono funzioni a <i>due argomenti</i> su un dominio T				DoubleFunction	LongToDoubleFunction
				LongFunction	LongToIntFunction
	ToDoubleFunction	ToIntFunction			ToLongFunction
	ToDoubleBiFunction	ToIntBiFunction			ToLongBiFunction
BooleanSupplier	ObjDoubleConsumer	ObjIntConsumer			ObjLongConsumer

NB: non sono previste interfacce funzionali standard per operatori a più di due argomenti;
nel caso, dovranno essere definiti dall'utente.



JAVA: LAMBDA EXPRESSION

«UNDER THE HOOD»

- In Java ogni lambda ha il tipo di un'opportuna interfaccia funzionale, in molti casi (ma non tutti!) *deducibile dal compilatore*

ESEMPIO

Java

La lambda expression

`(int x) -> x+1`

è un'istanza dell'interfaccia funzionale `IntUnaryOperator`

Poiché tale interfaccia dichiara il metodo:

`int applyAsInt(int operand)`

il compilatore *traduce la lambda expression in una classe anonima* che implementa tale metodo:

```
new IntUnaryOperator() {  
    public int applyAsInt(int x) { return x+1; }  
}
```

C#: DELEGATI

- In C# ogni lambda ha il **tipo** di un **opportuno delegato** – una sorta di interfaccia *chiamabile* tramite l'operatore `()`
- Esistono due *delegati standard* fondamentali:
 - **Func** $\langle T_1, T_2, \dots, T_N, T_{RET} \rangle$ esprime l'idea di *funzione a N argomenti* (dominio) di tipo $T_1, T_2, \dots, T_N$ e tipo di ritorno (codominio) T_{RET}
 - caso particolare: **Func** $\langle T_{RET} \rangle$ esprime l'idea di *funzione senza argomenti* (dominio void) e tipo di ritorno (codominio) T_{RET}
 - **Action** $\langle T_1, T_2, \dots, T_N \rangle$ esprime l'idea di *funzione a N argomenti* (dominio) di tipo $T_1, T_2, \dots, T_N$ e tipo di ritorno (codominio) vuoto
- A questi si aggiungono altri *delegati standard* di complemento
 - **Predicate** $\langle T_1, T_2, \dots, T_N \rangle$ esprime l'idea di *funzione a N argomenti* (dominio) di tipo $T_1, T_2, \dots, T_N$ e tipo di ritorno (codominio) *boolean*
 - **EventHandler** $\langle T \rangle$ esprime l'idea di *funzione a un argomento* usata per gestire eventi

ASSEGNARE E USARE LE LAMBDA

(1/6)

- Assegnare una lambda (versione con tutti i tipi specificati)

```
IntUnaryOperator f = (int x) -> x+3;  
Func<int,int>     f = (int x) => x+3;  
val f : Int => Int = x:Int => x+3;  
val f : (Int) -> Int = { x:Int -> x+3; }
```

In Java e C# non si può usare **var** nelle lambda: il target type deve essere *sempre esplicito*

Java

C#

Scala

Kotlin

- Assegnare una lambda (versione con type inference)

```
IntUnaryOperator f = x -> x+3;  
Func<int,int>     f = x => x+3;  
val f : Int => Int = x => x+3;  
val f = x:Int => x+3;  
val f : (Int) -> Int = { x -> x+3; }  
val f = { x:Int -> x+3; }
```

In Java e C# non si può usare **var** nelle lambda: il target type deve essere *sempre esplicito*

Java

C#

Scala

Kotlin



ASSEGNARE E USARE LE LAMBDA (2/6)

- Chiamare una lambda

```
System.out.println( f.applyAsInt(4) ); // brutto...  
Console.WriteLine( f(4) );  
println( f(4) );  
println( f(4) );
```

Java

C#

Scala

Kotlin

7

- in Java, purtroppo, la vera natura della lambda come «istanza di una classe anonima» emerge al momento della chiamata, rendendo *impossibile l'uso diretto dell'operatore ()*
- per questo, in Java le lambda non sono «veri tipi funzione»
- la necessità di doversi *ricordare il nome del metodo interno* con le ulteriori sotto-varianti dovute ai tipi primitivi peggiora le cose

ASSEGNARE E USARE LE LAMBDA

(3/6)

- Accettare una lambda come argomento

```
int myCalc(IntUnaryOperator f) {...}  
int myCalc(Func<int,int> f) {...}  
def myCalc(f: Int=>Int ) : Int = {...}  
fun myCalc(f: (Int)->Int ) : Int {...}
```

Da stabilire cosa
faccia **myCalc**

Java

C#

Scala

Kotlin

- Chiamare una funzione passando una lambda

```
var res = myCalc((int x) -> x+1); // o: x -> x+1  
var res = myCalc((int x) => x+1); // o: x => x+1  
val res = myCalc((x:Int) => x+1); // o: x => x+1  
val res = myCalc({x:Int -> x+1}); // o: {x -> x+1}
```

Java

C#

Scala

Kotlin

- Il valore di **res** dipende da cosa fa **myCalc**



ASSEGNARE E USARE LE LAMBDA (4/6)

- Una possibile definizione per myCalc:

```
int myCalc(IntUnaryOperator f) {  
    return f.applyAsInt(3) + 5; }  
int myCalc(Func<int,int> f) { return f(3)+5;}  
def myCalc(f: Int=>Int ) : Int = f(3)+5  
fun myCalc(f: (Int)->Int ) : Int = f(3)+5
```

Java

C#

Scala

Kotlin

- Corrispondenti risultati con diverse chiamate:

```
var res = myCalc(x -> x+1); // res = 9  
var res = myCalc(x -> 2*x-1); // res = 10  
var res = myCalc(x -> x*x-2); // res = 12
```

Java

- Il comportamento complessivo di **myCalc** dipende dalla funzione ricevuta come argomento, che le «inietta» comportamento

ASSEGNARE E USARE LE LAMBDA (5/6)

- Definire funzioni con dominio vuoto

f: void $\rightarrow$ R «funzione che genera un valore reale»

```
DoubleSupplier f = () -> Math.sqrt(2);
```

```
Func<double> f = () => Math.Sqrt(2);
```

```
val f : () => Double = () => Math.sqrt(2);
```

```
val f : () -> Double = { Math.sqrt(2.0); }
```

Java

C#

Scala

Kotlin

- Chiamare funzioni con dominio vuoto

```
System.out.println( f.getAsDouble() );
```

```
Console.WriteLine( f() );
```

```
println( f() );
```

```
println( f() );
```

In Java occorre conoscere il nome del metodo sottostante

Java

C#

Scala

Kotlin

ASSEGNARE E USARE LE LAMBDA

(6/6)

- Definire funzioni con codominio vuoto

$p: S \rightarrow \text{void}$ «procedura che, data una stringa, la stampa»

```
Consumer<String> p = s -> System.out.println(s);
```

Java

```
Action<string> p = (s) => Console.WriteLine(s);
```

C#

```
val p : String => Unit = (s) => println(s);
```

Scala

```
val p : (String) -> Unit = { s -> println(s); }
```

Kotlin

- Chiamare funzioni con codominio vuoto

```
p.accept("Hello World");
```

```
p("Hello World");
```

```
p("Hello World");
```

```
p("Hello World");
```

In Java occorre conoscere il nome del metodo sottostante

Java

C#

Scala

Kotlin



SCORCIATOIE PER LAMBDA

- Spesso le lambda sono molto semplici, limitandosi a:

1. richiamare un metodo esistente, senza argomenti

Esempio: `x -> x.getValue()`

2. fare semplici elaborazioni, senza argomenti

Esempio: `x -> x+1`

- In questi casi, i vari linguaggi rendono disponibili *scorciatoie* che permettono, secondo i casi, di:

- non scrivere proprio la lambda → *method reference*
- scrivere solo il lato destro → *parametri impliciti*

Java

Kotlin

Scala

Kotlin

METHOD REFERENCE

- Se la lambda si limita a *richiamare un metodo già esistente*, senza argomenti, si può sostituire con un *riferimento al metodo originario*: una **method reference**
- La sintassi si basa sul nuovo **operatore ::**

Java

Kotlin

NomeClasse::nomeMetodo

NomeIstanza::nomeMetodo

- la forma ***NomeClasse::nomemetodo*** è utilizzabile per ogni *metodo pubblico di una classe*
- la forma ***NomeIstanza::nomemetodo*** è utilizzabile anche per *metodi privati di specifici oggetti*: in particolare,
 - in Java **`this::method`** equivale a **`this -> this.method()`**
 - in Kotlin ***NomeIstanza*** può anche essere assente (**`=this`**)

METHOD REFERENCE: ESEMPI

- ESEMPI Java Kotlin

`Counter::getValue` sostituisce la lambda `x -> x.getValue()`

`Persona::getNome` sostituisce la lambda `p -> p.getNome()`

`Integer::toString` sostituisce la lambda `i -> i.toString()`

NB:

- non c'è l'operatore di chiamata `()`, solo il *nome del metodo*
- in Java ***NomeClasse*** può essere anche un array, come `Counter[]`
- occorre evitare ambiguità: la type inference deve poter determinare in modo univoco il tipo degli argomenti – altrimenti, conflitto!
 - esempio di conflitto: `Integer::toString`, perché in `Integer` convivono sia il *metodo* `toString` sia la *funzione statica* `toString`
 - non è detto che tale compresenza dia effettivamente luogo ad ambiguità: dipende dal contesto d'uso, caso per caso

PARAMETRI IMPLICITI

- Un'altra scorciatoia consiste nello *scrivere solo il lato destro della lambda*, evitando completamente il resto
- La sintassi si basa sull'uso di un apposito *placeholder*
 - in Scala, `_` sostituisce l'i-esimo argomento Scala
 - in Kotlin, `it` sostituisce il primo (e unico) argomento Kotlin
- ESEMPI Scala
 - `_`.getValue() sostituisce la lambda `c => c.getValue()`
 - `_+1` sostituisce la lambda `n => n+1`
- ESEMPI Kotlin
 - `{it.getValue() }` sostituisce la lambda `{c -> c.getValue() }`
 - `{it+1}` sostituisce la lambda `{n -> n+1}`

PARAMETRI IMPLICITI

- In Scala, il *placeholder* `_` può anche sostituire *contemporaneamente* più argomenti
- ESEMPI Scala

`val sum = (_: Int) + (_: Int)`
sostituisce la lambda $(x, y) \Rightarrow x + y$

`val mul : (Int, Int) => Int = _ * _`
*sostituisce la lambda $(x, y) \Rightarrow x * y$*

Il tipo degli argomenti va specificato o negli argomenti (come in `sum`), o nel tipo della lambda (come in `mul`)

ATTENZIONE: ogni occorrenza di `_` sostituisce *un diverso* argomento

`val pow : Int => Int = _ * _`

NO, errata: `_ * _` vale come $x * y$, non come $x * x$!